

4. 代码类资料会识别语言和文件类型，必要时结合 AST、LLM 或 AST+LLM 模式生成更有结构的代码摘要。

2.3 TreeBuilder 与 URI 落位

TreeBuilder 读取临时目录中的唯一文档根节点，清洗文件名，确定最终 URI。如果用户指定 `to`，则写入精确目标；如果指定 `parent`，则在父目录下创建资源；否则根据 `scope` 自动落到 `viking://resources`、`viking://user` 或 `viking://agent`。媒体资源会根据类型进入对应资源空间。若目标名称冲突，系统会自动追加 `_1`、`_2` 等后缀生成唯一 URI。

工程价值

临时目录 + 最终 URI 的两阶段设计，使解析失败不会污染正式知识库；同时也为增量同步、生命周期锁、`watch` 任务和处理状态回滚保留了清晰边界。

2.4 语义加工与向量化

资源树落位后，SemanticQueue 投递 SemanticMsg。SemanticProcessor 以目录为单位自底向上工作：先为叶子文件生成摘要，再汇总子目录摘要，生成当前目录的 `.overview.md` 和 `.abstract.md`，最后写入向量索引。系统支持增量更新：当 `target_uri` 已存在且与当前临时 URI 不同，会进入增量同步模式，只处理新增、修改和删除部分。

层级	内容	检索作用
L0 .abstract.md	高度压缩的节点摘要，约百 token 量级	用于快速向量召回、候选粗筛、低成本相关性判断
L1 .overview.md	目录级概览，汇总子节点摘要和结构信息	用于层级导航、重排、回答前定位上下文
L2 原始内容	文档正文、代码、表格转换文本、媒体摘要或原文件引用	用于最终引用、证据展开和生成答案

- 并发控制：SemanticProcessor 通过 `max_concurrent_llm` 控制摘要生成并发，避免模型服务过载。
- 熔断与重试：模型 API 出现临时错误时会重新入队，永久错误会记录失败并更新文档处理状态。
- 生命周期锁：资源同步与语义处理可持有 `subtree lock`，降低并发写入导致的不一致风险。
- 状态可观测：ResourceService、KnowledgeDocumentRegistry、QueueManager 和 telemetry 共同记录入库、等待、失败、队列耗时等指标。